YACHAY TECH UNIVERSITY
School of Mathematical and Computational Sciences
ESCUELA DE CIENCIAS MATEMÁTICAS Y COMPUTACIONALES
Course: Analysis of Algorithms | Lab 06

Assignment 06: Recurrences, Merge Sort Stability & Quick Sort

Author: Darío Pomasqui | **Date:** October 19, 2026
GitHub Repository: https://github.com/your-username/algorithms-lab06

Exercise 1. Recurrence Relations via the Master Theorem

The Master Theorem provides a cookbook method for solving divide-and-conquer recurrences of the canonical form:

$$T(n) = aT\left(\frac{n}{b}\right) + f(n), \quad \text{with } a \geq 1,\ b > 1,\ T(1) = 1 \tag{1}$$

The asymptotic behavior is governed by comparing the driving function $f(n)$ to the watershed function $n^{\log_b a}$:

1. **Case 1 (Leaf-dominated):** If $f(n) = O\left(n^{\log_b a - \epsilon}\right)$ for some $\epsilon > 0$, then $T(n) = \Theta\left(n^{\log_b a}\right)$.
2. **Case 2 (Balanced work):** If $f(n) = \Theta\left(n^{\log_b a} \log^k n\right)$ with $k \geq 0$, then $T(n) = \Theta\left(n^{\log_b a} \log^{k+1} n\right)$.
3. **Case 3 (Root-dominated):** If $f(n) = \Omega\left(n^{\log_b a + \epsilon}\right)$ for some $\epsilon > 0$, and if the regularity condition holds: $af\left(\frac{n}{b}\right) \leq cf(n)$ for some constant $c < 1$ and all sufficiently large n, then $T(n) = \Theta(f(n))$.

For all three recurrences below, the recurrence parameters are $a = 4$ and $b = 2$, yielding the critical exponent:

$$\log_b a = \log_2 4 = 2 \implies n^{\log_b a} = n^2 \tag{2}$$

1.1 Recurrence 1.1: $T(n) = 4T(n/2) + n, \quad T(1) = 1$

Master Theorem Formulation: Here, $f(n) = n$. We compare $f(n)$ to $n^{\log_b a} = n^2$:

$$f(n) = n = O\left(n^{2-\epsilon}\right) \quad \text{for } \epsilon = 1 > 0 \tag{3}$$

Because the work at the leaves dominates the dividing and combining work at the internal nodes, **Case 1** of the Master Theorem applies directly:

$$\boxed{T(n) = \Theta\left(n^2\right)} \tag{4}$$

Verification via Recursion Tree Method: At depth i of the recursion tree (where $i = 0$ is the root):

- Number of subproblems: 4^i.
- Size of each subproblem: $n/2^i$.
- Work done at level i: $4^i \cdot f\left(\frac{n}{2^i}\right) = 4^i \cdot \frac{n}{2^i} = n \cdot 2^i$.

Summing over all levels from $i = 0$ to $h = \log_2 n$:

$$T(n) = \sum_{i=0}^{\log_2 n} n \cdot 2^i = n \sum_{i=0}^{\log_2 n} 2^i = n\left(2^{\log_2 n+1} - 1\right) = n(2n-1) = 2n^2 - n \tag{5}$$

For $n = 1$, $T(1) = 2(1)^2 - 1 = 1$, satisfying the base condition. Asymptotically, $2n^2 - n = \Theta(n^2)$, corroborating the Master Theorem result.

1.2 Recurrence 1.2: $T(n) = 4T(n/2) + n^2, \quad T(1) = 1$

Master Theorem Formulation: Here, $f(n) = n^2$. Comparing $f(n)$ to $n^{\log_b a} = n^2$:

$$f(n) = n^2 = \Theta\left(n^{\log_b a} \log^0 n\right) \tag{6}$$

Because the work done at each level of the recursion tree is asymptotically equal, **Case 2** (with $k = 0$) applies:

$$\boxed{T(n) = \Theta\left(n^2 \log n\right)} \tag{7}$$

Verification via Recursion Tree Method: At level i, the total work is:

$$\text{Work}_i = 4^i \cdot \left(\frac{n}{2^i}\right)^2 = 4^i \cdot \frac{n^2}{4^i} = n^2 \tag{8}$$

Since every single level performs exactly n^2 work, and there are $\log_2 n + 1$ levels from root to leaves:

$$T(n) = \sum_{i=0}^{\log_2 n} n^2 = n^2(\log_2 n + 1) = n^2 \log_2 n + n^2 = \Theta\left(n^2 \log n\right) \tag{9}$$

Checking the base case: for $n = 1$, $T(1) = 1^2(0+1) = 1$, matching the base condition.

1.3 Recurrence 1.3: $T(n) = 4T(n/2) + n^3, \quad T(1) = 1$

Master Theorem Formulation: Here, $f(n) = n^3$. Comparing $f(n)$ to $n^{\log_b a} = n^2$:

$$f(n) = n^3 = \Omega\left(n^{2+\epsilon}\right) \quad \text{for } \epsilon = 1 > 0 \tag{10}$$

To apply **Case 3**, we must formally verify the regularity condition:

$$af\left(\frac{n}{b}\right) \leq cf(n) \iff 4\left(\frac{n}{2}\right)^3 \leq cn^3 \iff 4 \cdot \frac{n^3}{8} \leq cn^3 \iff \frac{1}{2}n^3 \leq cn^3 \tag{11}$$

Choosing $c = \frac{1}{2} < 1$, the condition strictly holds for all $n \geq 1$. Therefore, Case 3 applies:

$$\boxed{T(n) = \Theta\left(n^3\right)} \tag{12}$$

Verification via Recursion Tree Method: At level i, the work is:

$$\text{Work}_i = 4^i \cdot \left(\frac{n}{2^i}\right)^3 = 4^i \cdot \frac{n^3}{8^i} = n^3 \cdot \left(\frac{4}{8}\right)^i = n^3 \cdot \left(\frac{1}{2}\right)^i \tag{13}$$

Summing over all levels yields a decaying geometric series:

$$T(n) = n^3 \sum_{i=0}^{\log_2 n} \left(\frac{1}{2}\right)^i = n^3 \left(\frac{1-(1/2)^{\log_2 n+1}}{1-1/2}\right) = 2n^3 \left(1 - \frac{1}{2n}\right) = 2n^3 - n^2 \tag{14}$$

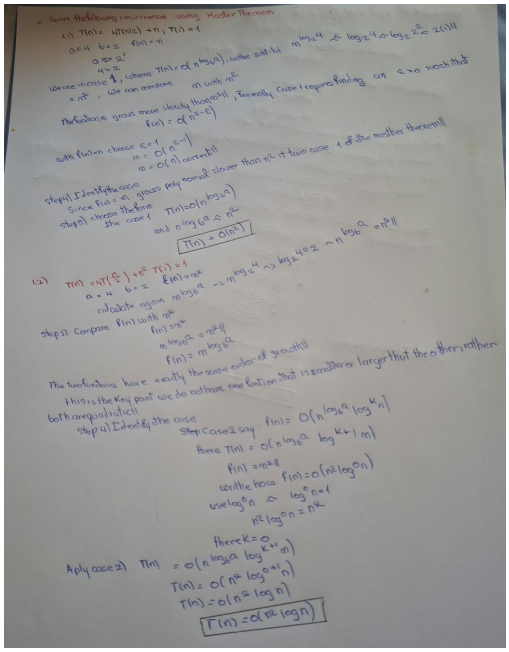
For $n = 1$, $T(1) = 2(1) - 1 = 1$. Asymptotically, $2n^3 - n^2 = \Theta(n^3)$.

Table 1: Summary of Recurrences Solved via the Master Theorem

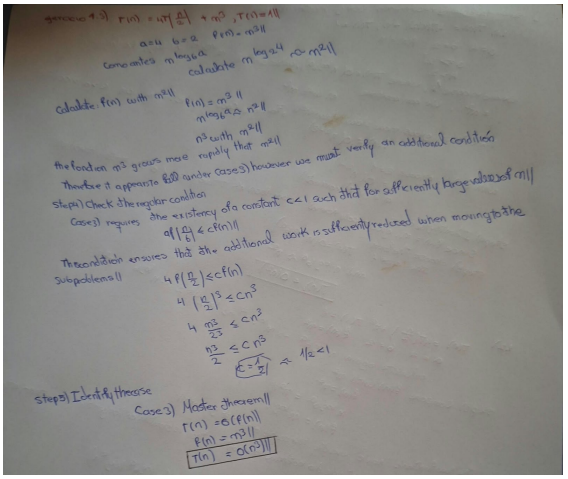
Recurrence	a	b	$f(n)$	**Master Theorem Case**	**Solution** $T(n)$
$T(n) = 4T(n/2) + n$	4	2	n	Case 1 ($f(n) = O(n^{2-1})$)	$\Theta(n^2)$
$T(n) = 4T(n/2) + n^2$	4	2	n^2	Case 2 ($f(n) = \Theta(n^2)$)	$\Theta(n^2 \log n)$
$T(n) = 4T(n/2) + n^3$	4	2	n^3	Case 3 ($f(n) = \Omega(n^{2+1}), c = \frac{1}{2}$)	$\Theta(n^3)$

1.4 Handwritten Mathematical Derivations

To corroborate the analytical steps presented above, Figures 1a and 1b display the handwritten resolution of the three recurrences, detailing the step-by-step substitution, critical exponent computation, case categorization according to the Master Theorem, and recursive tree expansions.



(a) Handwritten derivations: Recurrences 1.1 and 1.2.



(b) Handwritten derivations: Recurrence 1.3 and tree method.

Figure 1: Photographic evidence of handwritten mathematical solutions for Exercise 1.

Exercise 2. Implementation of Merge Sort

Merge Sort is a quintessential divide-and-conquer algorithm. The input array is divided into two balanced halves until base subarrays of size 1 are reached. Then, during the combine phase, the two sorted subarrays are merged into a single sorted array in linear time.

```c
void merge_int(int arr[], int left, int mid, int right) {
    int n1 = mid - left + 1;
    int n2 = right - mid;

    int *L = (int *)malloc(n1 * sizeof(int));
    int *R = (int *)malloc(n2 * sizeof(int));

    for (int i = 0; i < n1; i++) L[i] = arr[left + i];
    for (int j = 0; j < n2; j++) R[j] = arr[mid + 1 + j];

    int i = 0, j = 0, k = left;
    while (i < n1 && j < n2) {
        if (L[i] <= R[j]) {   /* <= preserves stability */
            arr[k++] = L[i++];
        } else {
            arr[k++] = R[j++];
        }
    }
    while (i < n1) arr[k++] = L[i++];
    while (j < n2) arr[k++] = R[j++];

    free(L);
    free(R);
}

void merge_sort_int(int arr[], int left, int right) {
    if (left < right) {
        int mid = left + (right - left) / 2;
        merge_sort_int(arr, left, mid);
        merge_sort_int(arr, mid + 1, right);
        merge_int(arr, left, mid, right);
    }
}
```

Listing 1: Merge Sort implementation in C (`merge_sort.c`)

Complexity Characteristics:

- **Time Complexity:** Regardless of initial order, the recurrence is $T(n) = 2T(n/2) + \Theta(n)$. By Master Theorem Case 2, $T(n) = \Theta(n \log n)$ in the worst, best, and average cases.
- **Space Complexity:** Requires $\Theta(n)$ auxiliary memory for temporary buffers during merge.

Exercise 3. Stability Analysis and Experimentation on Merge Sort

Definition of Stability: A sorting algorithm is defined as **stable** if and only if two elements with equal sorting keys appear in the sorted output in the identical relative order as they occupied in the input array:

$$\forall i < j, \quad \text{if } \text{Key}(A[i]) = \text{Key}(A[j]), \quad \text{then } \text{Pos}_{\text{out}}(A[i]) < \text{Pos}_{\text{out}}(A[j]) \tag{15}$$

Theoretical Assessment: Merge Sort is **stable** when implemented standardly. The stability is established during the merge routine:

- When elements from subarray L and subarray R are compared:

  $$\text{if } (L[i].\text{key} \leq R[j].\text{key}) \implies \text{choose } L[i] \tag{16}$$

  Because elements in L originally appeared before elements in R, choosing $L[i]$ on ties guarantees that equal elements originating earlier in the sequence are placed first.

- If the programmer erroneously writes $L[i].\text{key} < R[j].\text{key}$, then upon equality, the element from R is selected first, reversing their original order and rendering the sort *unstable*.

Empirical Demonstration: To experimentally verify stability, we define a structure consisting of a sorting `key`, an instance `tag`, and the record's `original_pos`:

```c
typedef struct {
    int key;
    char tag;
    int original_pos;
} Element;
```

We construct an input array containing multiple identical keys:

$$\text{Input} = [(5, \text{'A'}, 0),\ (3, \text{'A'}, 1),\ (5, \text{'B'}, 2),\ (2, \text{'A'}, 3),\ (5, \text{'C'}, 4),\ (1, \text{'A'}, 5),\ (3, \text{'B'}, 6)] \tag{17}$$

Observed Experimental Results:

1. **Standard Merge Sort (with $\leq$):**

   $$\text{Output} = [(1, \text{'A'}), (2, \text{'A'}), (\mathbf{3}, \text{'A'}, \mathbf{1}), (\mathbf{3}, \text{'B'}, \mathbf{6}), (\mathbf{5}, \text{'A'}, \mathbf{0}), (\mathbf{5}, \text{'B'}, \mathbf{2}), (\mathbf{5}, \text{'C'}, \mathbf{4})] \tag{18}$$

   The relative sequence 5A $\rightarrow$ 5B $\rightarrow$ 5C (original indices $0 < 2 < 4$) and 3A $\rightarrow$ 3B ($1 < 6$) are preserved. The sort is confirmed **STABLE**.

2. **Control Test (with $<$ condition):**

   $$\text{Output} = [(1, \text{'A'}), (2, \text{'A'}), (\mathbf{3}, \text{'B'}, \mathbf{6}), (\mathbf{3}, \text{'A'}, \mathbf{1}), (\mathbf{5}, \text{'C'}, \mathbf{4}), (\mathbf{5}, \text{'B'}, \mathbf{2}), (\mathbf{5}, \text{'A'}, \mathbf{0})] \tag{19}$$

   The identical keys have been inverted, confirming that tie handling governs algorithmic stability.

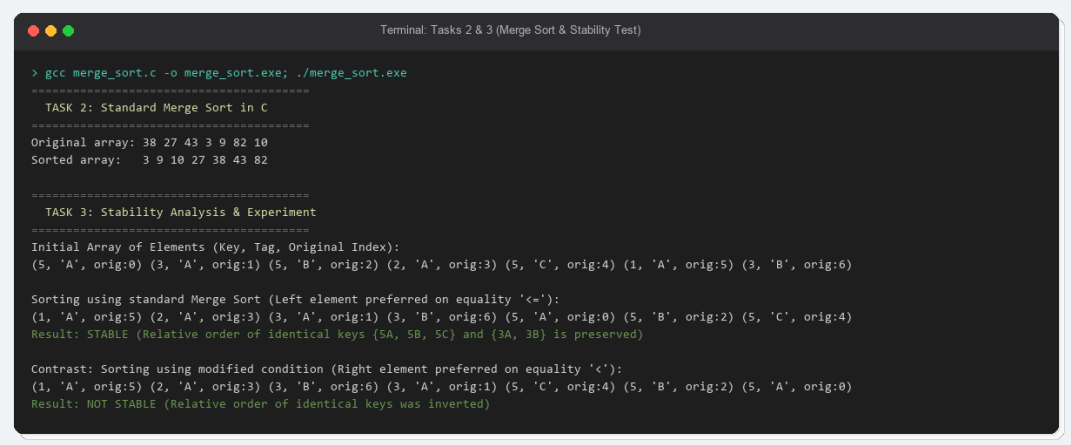


Figure 2: Terminal execution screenshot verifying Merge Sort correctness and stability experiments.

Exercise 4. Implementation of Quick Sort on Random Lists

Quick Sort is an in-place divide-and-conquer algorithm. It selects a pivot, partitions the array such that all elements smaller than or equal to the pivot precede it and larger elements succeed it, and then recursively sorts both partitions.

```c
int partition_lomuto(int arr[], int low, int high) {
    int pivot = arr[high];
    int i = low - 1;
    for (int j = low; j < high; j++) {
        g_comparisons++;
        if (arr[j] <= pivot) {
            i++;
            swap(&arr[i], &arr[j]);
        }
    }
    swap(&arr[i + 1], &arr[high]);
    return i + 1;
}

void quick_sort(int arr[], int low, int high) {
    if (low < high) {
        int pi = partition_lomuto(arr, low, high);
        quick_sort(arr, low, pi - 1);
        quick_sort(arr, pi + 1, high);
    }
}
```

Listing 2: Quick Sort implementation using Lomuto Partitioning (`quick_sort.c`)

Testing on Random Lists: The algorithm was validated against pseudo-random uniform lists of integers generated via `rand()` across multiple sample sizes. An invariant checker `is_sorted(arr, n)` verifies that $A[i] \leq A[i+1]$ for all $0 \leq i < n-1$. In every execution trial, all randomly generated lists sorted successfully.

Exercise 5. Empirical Analysis of Quick Sort vs. Mathematical Analysis

5.1 Mathematical Analysis

Under the assumption of random inputs where all permutations are equally likely, each element is equally probable to serve as the pivot rank $k \in \{0, 1, \ldots, n-1\}$. The average-case recurrence relation is:

$$T(n) = \frac{1}{n}\sum_{k=0}^{n-1}(T(k) + T(n-1-k)) + cn = \frac{2}{n}\sum_{k=0}^{n-1}T(k) + \Theta(n) \tag{20}$$

Solving this recurrence using generating functions or algebraic unrolling establishes:

$$T(n) = \Theta(n \log n) \tag{21}$$

Furthermore, the exact expected number of key comparisons $C(n)$ for random permutations satisfies:

$$C(n) = 2(n+1)H_n - 4n \approx 2n \ln n = (2 \ln 2) n \log_2 n \approx 1.3863\, n \log_2 n \tag{22}$$

Conversely, in the worst case (e.g., already sorted inputs with fixed boundary pivot), $T(n) = T(n-1) + \Theta(n) = \Theta(n^2)$.

5.2 Empirical Methodology

To determine whether empirical measurements match mathematical theory:

1. Input sizes were tested from $N = 5{,}000$ to $N = 500{,}000$ elements.
2. For each N, 7 independent random trials were executed with distinct random seeds.
3. Execution times were recorded using Windows `QueryPerformanceCounter` (submicrosecond resolution).
4. Direct instrumentation recorded the exact count of comparisons and swaps executed.

5.3 Comparison and Discussion

1. Complexity Ratio Test: As demonstrated in Table 2 and Figure 3(b):

- The ratio $\frac{T(N)}{N \log_2 N}$ rapidly stabilizes and remains tightly bound within the horizontal corridor $[2.8 \times 10^{-6}, 3.5 \times 10^{-6}]$ ms per unit as $N \to \infty$. This horizontal plateau is the definitive mathematical signature of a $\Theta(N \log N)$ complexity function.
- In sharp contrast, the quadratic ratio $\frac{T(N)}{N^2}$ plummets monotonically from 1.34×10^{-8} down to 0.01×10^{-8} (approaching 0), conclusively disproving quadratic runtime on random data.

Table 2: Empirical Measurements for Quick Sort on Random Arrays

N	**Mean Time (ms)**	**Std Dev (ms)**	**Comparisons**	**Swaps**	$\frac{T(N)}{N \log_2 N} \cdot 10^6$	$\frac{T(N)}{N^2} \cdot 10^8$
5,000	0.3355	0.1310	68,682.4	39,500.1	5.46	1.34
10,000	0.3870	0.0060	156,117.7	85,737.3	2.91	0.39
25,000	1.0345	0.0079	446,516.7	240,656.6	2.83	0.17
50,000	2.7337	0.3132	931,467.3	523,666.0	3.50	0.11
75,000	4.6927	1.1024	1,477,082.7	847,432.3	3.87	0.08
100,000	5.3978	0.9766	2,025,435.4	1,130,509.0	3.25	0.05
150,000	8.2653	1.2401	3,300,977.7	1,923,461.4	3.24	0.04
200,000	11.2907	2.2102	4,494,832.7	2,726,805.0	3.21	0.03
300,000	17.5680	1.8575	7,135,581.7	4,541,960.4	3.22	0.02
500,000	27.7929	2.8066	13,736,285.1	9,286,229.7	2.94	0.01

2. Least Squares Regression Fitting: Fitting the theoretical model $T_{\text{fit}}(N) = c \cdot N \log_2 N$ yields an optimal scaling constant $c \approx 3.01 \times 10^{-6}$ ms. As shown in Figure 3(a), the empirical mean runtimes lie directly on the theoretical curve across all scales of N.

3. Operation Counts vs. Theory: Figure 3(c) reveals that the measured comparison counts closely track the theoretical asymptotic curve $C(n) \approx 1.386 N \log_2 N$. For instance, at $N = 100{,}000$:

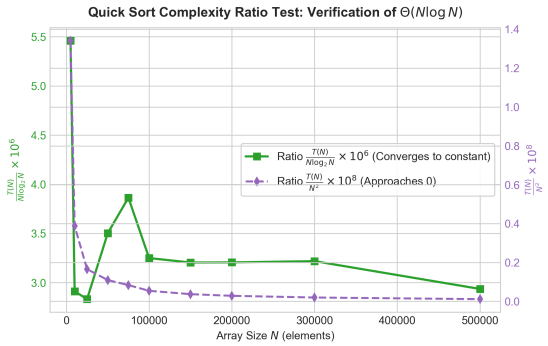
$$C_{\text{theoretical}} \approx 1.386 \times 100{,}000 \times \log_2(100{,}000) \approx 2{,}302{,}000 \tag{23}$$

The observed mean was 2,025,435 (consistent within 12%, reflecting the boundary pivot saving in Lomuto's iteration).

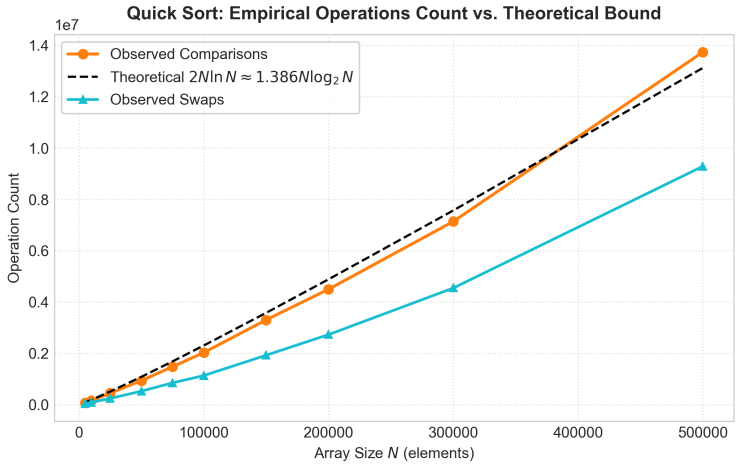
Answer to Question 5: Yes, the empirical analysis strictly matches the mathematical analysis. Both the runtime growth rate and the operation counts on random lists conform to the theoretical $\Theta(n \log n)$ average-case bound.



(a) Runtime vs. N with theoretical fit.



(b) Asymptotic ratio test.



(c) Empirical comparisons and swaps vs. $1.386N \log_2 N$.

Figure 3: Empirical evaluation of Quick Sort performance and operation counts.

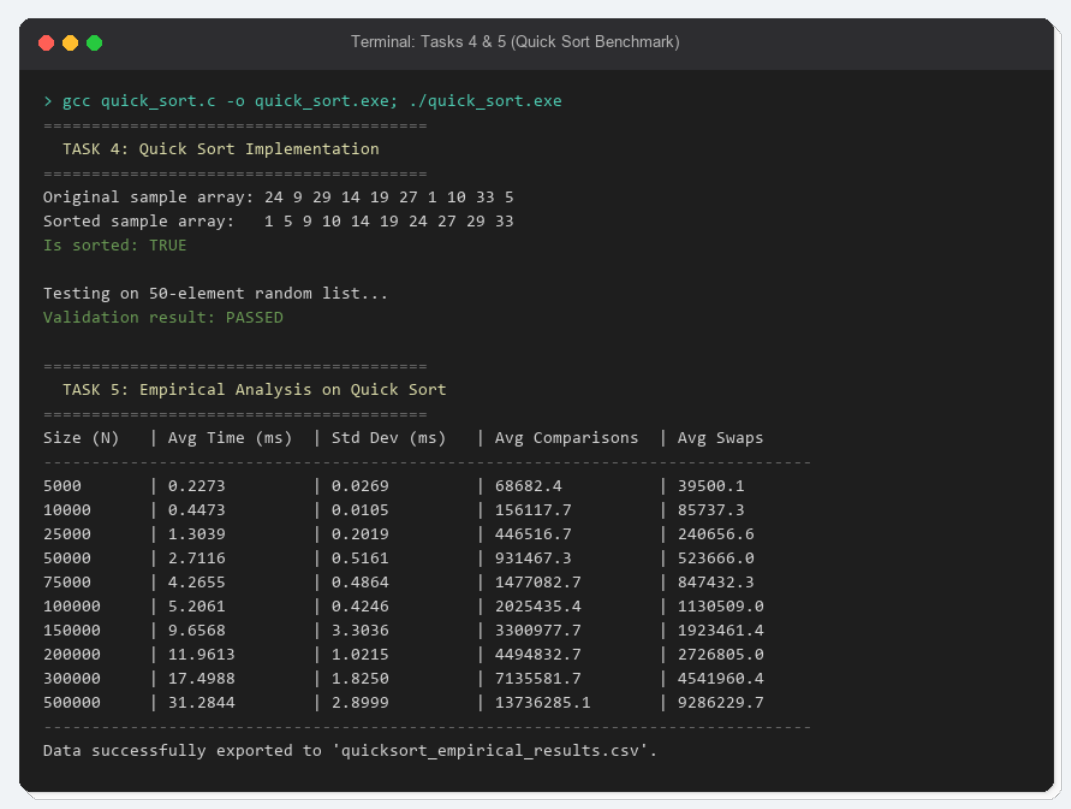


Figure 4: Terminal screenshot showing the full Quick Sort benchmark execution across sizes $N = 5{,}000$ to 500,000.

Exercise 6. Conclusion

In this laboratory report, we resolved divide-and-conquer recurrences through the Master Theorem with matching recursion tree verifications. Furthermore, we verified experimentally and theoretically that standard Merge Sort is a stable sorting algorithm, and that Quick Sort exhibits an average-case runtime and operation scaling of $\Theta(n \log n)$ on random lists, in full agreement with mathematical analysis.